OOPS Project Report

TEAM 49

HANGMAN QUIZ GAME

CS23B1002, CS23B1018, CS23B2009, CS23I1016

IIITDM Kancheepuram

Introduction

This code implements the classic word-guessing game **Hangman** in C++, showcasing several important Object-Oriented Programming (OOP) concepts. The player attempts to guess a word based on a series of clues, with a limited number of incorrect guesses allowed. The game uses ASCII art to display the hangman's progress based on incorrect guesses, and the player has the option to receive hints during the game. The code demonstrates the following OOP principles:

- **Encapsulation**: Game elements like the game logic, questions, and player states are encapsulated within distinct classes (Hangman, Question, QuestionManager).
- **Abstraction**: Each class abstracts specific responsibilities, such as handling questions, game logic, and user input, simplifying the game's structure.
- **Constructor and Destructor**: Classes use constructors for initialization (e.g., setting up the question list) and destructors for cleanup (e.g., displaying a farewell message).
- **Function Pointers**: Function pointers are used to manage dynamic behavior, such as selecting random questions and checking user guesses.
- **Operator Overloading**: The `<<`, `>>`, and `==` operators are overloaded to handle custom input/output operations, like displaying the game state and comparing objects.
- **Composition**: The Hangman class uses composition to manage instances of the `QuestionManager` class, which stores the questions and answers.
- **Templates**: The `QuestionManager` class is implemented as a template to handle various types of questions, making the code more reusable and flexible.
- **Static Members**: The `BaseGame` class uses static members to track the total number of games played and wins across all instances.
- **Inheritance and Polymorphism**: The `Hangman` class inherits from the `BaseGame` class and overrides the `play()` function to implement game-specific logic.
- **File Handling**: The game supports loading questions from a file, ensuring flexibility in question sets.
- **Namespace Usage**: The game code uses the `HangmanGame` namespace to logically group related classes and functions.
- **Error Handling**: The game includes exception handling for potential runtime errors, such as file loading failures or empty question sets.
- This game uses multiple OOP principles and advanced C++ features to create an engaging word-guessing experience. Each class is designed for a specific function, ensuring modularity and maintainability. The use of function and operator overloading, along with function pointers, adds flexibility, while encapsulation ensures each component handles its own functionality.

Libraries

Code Snippet

```
#include <iostream>
#include <string>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <memory>
#include <fstream>
#include <stdexcept>
```

The code snippet demonstrates the following libraries:

- **include iostream**: Manages input and output operations, such as printing messages and prompts to the console.
- **include string**: Facilitates the manipulation of strings, allowing for storage and modification of text data, such as the guessed letters in the game.
- **include vector**: Offers a dynamic array using `std::vector`, which is used to store and manage a list of questions in the game.
- **include cstdlib**: Provides general-purpose functions, including `rand()` for generating random numbers, and `srand()` for setting a seed for randomness.
- **include ctime**: Enables time-related operations, such as obtaining the current time to seed the random number generator for improved randomness.
- **include memory**: Facilitates dynamic memory management, often used for managing memory in the game, particularly with dynamic objects or pointers.
- **include fstream**: Provides file handling functionality, such as reading from and writing to files. In this case, it is used to load questions from a file.
- **include stdexcept**: Supplies standard exceptions, which can be used to handle and report runtime errors.

Classes :

Question Class

The `Question` class is used to represent a question with its associated answer. It holds the question prompt and the correct answer for use in the game. The class provides methods to retrieve the question and answer.

Code Snippet

```
class Question {
private:
    string prompt;
    string answer;

public:
    Question(const string& p, const string& a) : prompt(p), answer(a) {}

    string getPrompt() const { return prompt; }
    string getAnswer() const { return answer; }

    // Overload the << operator to display the question
    friend ostream& operator<<(ostream& os, const Question& q) {
        os << "Question: " << q.prompt;
        return os;
    }
};
```

Functions used in this class:

- `Question(const string& p, const string& a)`: This is the constructor of the `Question` class. It initializes the question and answer fields with the provided prompt and answer.
- `getPrompt() const`: A getter function that returns the question prompt.
- `getAnswer() const`: A getter function that returns the answer to the question.
- `operator<<`: This is the overloaded stream insertion operator. It is used to print the question prompt when the `Question` object is passed to an output stream, such as `cout`.

QuestionManager Class

The `QuestionManager` class is a template class that manages a collection of `Question` objects. It allows adding questions, selecting random questions for gameplay, and retrieving the number of stored questions.

Code Snippet

```
template <typename T>
class QuestionManager {
private:
    vector<T> questions;

public:
    void addQuestion(const T& question) {
        questions.push_back(question);
    }

    T getRandomQuestion() {
        srand(static_cast<unsigned>(time(0)));
        int index = rand() % questions.size();
        return questions[index];
    }

    size_t size() const {
        return questions.size();
    }

    // Overload the << operator to display all questions
    friend ostream& operator<<(ostream& os, const QuestionManager<T>& qm) {
        for (const auto& question : qm.questions) {
            os << question << endl;
        }
        return os;
    }
};
```

Functions used in this class:

- `addQuestion(const T& question)`: Adds a new question to the `questions` vector.
- `getRandomQuestion()`: Returns a random question from the list of questions using the `rand()` function to generate a random index.
- `size() const`: Returns the size (number of questions) in the `questions` vector.
- `operator<<`: This is the overloaded stream insertion operator. It is used to display all the questions in the `QuestionManager` object when it is passed to an output stream, such as `cout`.

Hangman Class

The `Hangman` class manages the overall game flow. It interacts with the `QuestionManager` to retrieve random questions and handles the game logic, such as checking guesses, providing hints, and maintaining game statistics.

Code Snippet

```
class Hangman {
private:
    QuestionManager<Question> questionManager;
    string guess_word;
    string user_guess;
    char guess;
    int correct_guess;
    int incorrect_guess;
    int word_length;
    int hints_used;
    int remainingHints;

    static int totalGamesPlayed;
    static int totalWins;

    void (Hangman::*selectQuestion)();
    bool (Hangman::*guessChecker)();

    void selectRandomQuestion();
    bool checkGuess();
    void displayLevel(int level) const;

public:
    Hangman();
    ~Hangman();

    void play();
    bool won;

    static void displayStats();

    friend ostream& operator<<(ostream& os, const Hangman& game);
    friend istream& operator>>(istream& is, Hangman& game);

    bool operator==(const Hangman& other) const;
    void provideHint();

    friend void displayGameInfo(const Hangman& game);
};
```

Functions used in this class:

- `selectRandomQuestion()`: Selects a random question using the `QuestionManager`.

- `checkGuess()`: Compares the player's guess to the correct answer and returns whether it is correct.
- `displayLevel(int level) const`: Displays the current level or status of the hangman based on the number of incorrect guesses.
- `play()`: Starts and runs the game loop, allowing the user to make guesses and interact with the game.
- `displayStats()`: Displays statistics about the number of games played and won.
- `provideHint()`: Gives the player a hint by revealing part of the word or some other help.
- Operator overloading functions (`<<`, `>>`, and `==`) are defined to handle input/output for the game and comparison between two game objects.
- `displayGameInfo()`: This friend function displays specific game information, such as the word to guess, the number of incorrect guesses, and the remaining hints. It can access the private members of the `Hangman` class.

Friend Function Definition

Here is the definition of the `displayGameInfo()` friend function, which accesses private members of the `Hangman` class.

Code Snippet

```
void displayGameInfo(const Hangman& game) {
    cout << "Game Information: \n";
    cout << "Word to Guess: " << game.guess_word << endl;
    cout << "Incorrect Guesses: " << game.incorrect_guess << endl;
    cout << "Remaining Hints: " << game.remainingHints << endl;
}
```

Functions for Displaying Instructions

Code Snippet

```
void displayInstructions() {
    cout << "\n";
    cout << "+-------------------------------+\n";
    cout << "|         Instructions           |\n";
    cout << "+-------------------------------+\n";
    cout << "| Guess the word by entering    |\n";
    cout << "| letters. The number of wrong  |\n";
    cout << "| guesses is limited. You can    |\n";
    cout << "| also request hints during     |\n";
    cout << "| the game.                     |\n";
    cout << "+-------------------------------+\n";
}
```

Functions used:

- `displayInstructions()`: Displays a set of instructions to the user about how to play the Hangman game, including guessing letters and using hints.

Operator Overloading

The code uses operator overloading to enhance input/output operations and compare game states.

Code Snippet

```
ostream& operator<<(ostream& os, const Hangman& game) {
    os << "Guess Word: " << game.guess_word << endl;
    os << "Incorrect Guesses: " << game.incorrect_guess << endl;
    return os;
}

istream& operator>>(istream& is, Hangman& game) {
    cout << "Enter a letter to guess: ";
    is >> game.guess;
    return is;
}

bool Hangman::operator==(const Hangman& other) const {
    return this->guess_word == other.guess_word;
}
```

Functions used:

- `operator<<`: Overloads the `<<` operator to allow displaying the current state of the game, including the guessed word and the number of incorrect guesses.
- `operator>>`: Overloads the `>>` operator to allow input of the player's guess.
- `operator==`: Compares two `Hangman` objects to check if their guess words are identical.

Static Function for Displaying Statistics

The `displayStats()` static function is used to display the total number of games played and the total number of games won across all instances of the `Hangman` class.

Code Snippet

```
void Hangman::displayStats() {
    cout << "Total Games Played: " << totalGamesPlayed << endl;
    cout << "Total Wins: " << totalWins << endl;
}
```

Constructor and Destructor for Hangman Class

The constructor initializes game variables such as the word to guess, the number of incorrect guesses, and the available hints. The destructor is used for any necessary cleanup.

Code Snippet

```
Hangman::Hangman() : guess_word(""), user_guess(""), guess(0), correct_guess(0),
                     incorrect_guess(0), word_length(0), hints_used(0),
                     remainingHints(3), won(false) {
    // Initialization code
}

Hangman::~Hangman() {
    // Destructor code (if any needed)
}
```

The Play Function

The `play()` function starts the game loop, handling user input and checking guesses against the correct answer.

Code Snippet

```
void Hangman::play() {
    while (incorrect_guess < 6 && !won) {
        // Main game loop
        cout << "Current Guess: " << user_guess << endl;
        cout << "Enter a letter to guess: ";
        cin >> guess;
        if (checkGuess()) {
            correct_guess++;
        } else {
            incorrect_guess++;
        }
        displayLevel(incorrect_guess);
        if (correct_guess == word_length) {
            won = true;
        }
    }
    if (won) {
        cout << "You won!" << endl;
        totalWins++;
    } else {
        cout << "Game Over!" << endl;
    }
    totalGamesPlayed++;
}
```

Select Random Question Function

The `selectRandomQuestion()` function picks a random question from the available questions in the `QuestionManager`.

Code Snippet

```
void Hangman::selectRandomQuestion() {
    Question randomQuestion = questionManager.getRandomQuestion();
    guess_word = randomQuestion.getAnswer();
    word_length = guess_word.length();
    user_guess = string(word_length, '_');
}
```

Check Guess Function

The `checkGuess()` function verifies if the player's guessed letter is part of the word.

Code Snippet

```
bool Hangman::checkGuess() {
    bool correct = false;
    for (int i = 0; i < word_length; i++) {
        if (guess_word[i] == guess) {
            user_guess[i] = guess;
            correct = true;
        }
    }
    return correct;
}
```

Display Level Function

The `displayLevel()` function shows the current hangman state based on incorrect guesses.

Code Snippet

```
void Hangman::displayLevel(int level) const {
    // Display different hangman levels based on the number of
    incorrect guesses
    switch (level) {
        case 1: cout << "O" << endl; break;
        case 2: cout << "O\n|" << endl; break;
        case 3: cout << "O\n/|" << endl; break;
        case 4: cout << "O\n/|\\" << endl; break;
        case 5: cout << "O\n/|\\\n/" << endl; break;
        case 6: cout << "O\n/|\\\n/ \\" << endl; break;
        default: cout << "No wrong guesses yet!" << endl; break;
    }
}
```

Output Screenshots

```
PS C:\OOPS\PROJECT> g++ main.cpp .\project.cpp
PS C:\OOPS\PROJECT> ./a.exe

+-------------------------------+
|          Main Menu            |
+-------------------------------+
| (1) -> Play game              |
| (2) -> How to play            |
| (3) -> Exit                   |
+-------------------------------+
Enter your choice: 2

How to Play Hangman:
1. You will be given a question or a hint related to a word.
2. Type one letter at a time to guess the word.
3. If the letter is correct, it will be revealed in the word.
4. If you guess all letters correctly, you win!
5. You lose if you make 6 incorrect guesses.
6. You can type '!' to use a hint if you are stuck (limited to 2 hints).
```

Figure 1: INSTRUCTIONS

```
PS C:\OOPS\PROJECT> g++ main.cpp .\project.cpp
PS C:\OOPS\PROJECT> ./a.exe

+-------------------------------+
|          Main Menu            |
+-------------------------------+
| (1) -> Play game              |
| (2) -> How to play            |
| (3) -> Exit                   |
+-------------------------------+
Enter your choice: 1
Question: What is the mechanism where a new class can inherit the properties and behaviors of another class?
Guess Word: __________
Available Letters: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Enter your guess (or type '!' for a hint): i
Correct guess!
Hangman Level: 0

                  _____
                  |   |
                  |
                  |
                  |
                  |
               ---------Guess Word: i____i_____
```

Figure 2: START

```
Enter your guess (or type '!' for a hint): c
Correct guess!
Hangman Level: 5

                  -----
                  |   |
                  |   O
                  |  /|\
                  |  /
                  |
                ---------
WW      WW     II     NN      NN     NN     NN     EEEEEEEE     RRRRRR
WW      WW     II     NN N    NN     NNN    NN     EE           RR   RR
WW      WW     II     NN  N   NN     NN N   NN     EEEEEEEE     RRRRRR
 WW WW WW      II     NN   N  NN     NN  N  NN     EE           RR  RR
   W   W       II     NN    N NN     NN   N NN     EEEEEEEE     RR   RR


Congratulations! You've guessed the word!

Total Games Played: 3
Total Wins: 1
```

Figure 3: FINISH(WIN)

```
Enter your guess (or type '!' for a hint): W
Incorrect guess!
Hangman Level: 6

                  -----
                  |   |
                  |   O
                  |  /|\
                  |  / \
                  |
                ---------
LL             000000      SSSSSSS     EEEEEEEE
LL            00    00    SS          EE
LL            00    00     SSSSSSS    EEEEEEEE
LL            00    00          SS    EE
LLLLLLL        000000      SSSSSSS     EEEEEEEE


You've been hanged! The correct word was: abstraction

Total Games Played: 1
Total Wins: 0
```

Figure 4: FINISH(LOSE)

Conclusion

The Hangman Quiz Game is an excellent example of applying Object-Oriented Programming principles to create an interactive and maintainable C++ application. By leveraging encapsulation, inheritance, polymorphism, and other OOP techniques, the game allows for flexible

game management and user interaction. The modularity of the code ensures that different components, such as the question management system and game logic, can be easily updated or extended. Overall, this project demonstrates the power of C++ and OOP in building engaging and scalable applications.

References :

Textbook: *C++ How to Program* by Deitel and Deitel